FIAP ON

DATA VIZ & DATA SCIENCE

MODELAGEM DE DADOS

15

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Exemplos de tipos de estruturas de dados 10
Figura 2 – Os 5 V's do Big Data 12
Figura 3 – Ciclo de vida de soluções baseadas em Ciência de Dados 13
Figura 4 – Separação de dados em treino e teste.....15
Figura 5 – Separação de dados em treino, validação e teste 16
Figura 6 – Fragmento de uma base de dados real.....21
Figura 7 – Exemplo de mamografia com destaque para dois nódulos.....22
Figura 8 – Exemplos lúdicos de overfitting.....24
Figura 9 – Definição de modelo ideal 25
Figura 10 – Resultado do código-fonte 1 27
Figura 11 – Resultado do código-fonte 2 28
Figura 12 – Resultado do código-fonte 3 28
Figura 13 – Resultado do código-fonte 4 28
Figura 14 – Resultado do código-fonte 5 29
Figura 15 – Resultado do código-fonte 6 29
Figura 16 – Alterando tamanhos de arrays NumPy.....30
Figura 17 – Resultado do código-fonte 7 31
Figura 18 – Resultado do código-fonte 8 31
Figura 19 – Estruturas de dados da biblioteca Pandas 32
Figura 20 – Resultado do código-fonte 9 33
Figura 21 – Resultado do código-fonte 10 33
Figura 22 – Resultado do código-fonte 11 34
Figura 23 – Resultado do código-fonte 12 34
Figura 24 – Resultado do código-fonte 13 35
Figura 25 – Resultado do código-fonte 14 36
Figura 26 – Resultado do código-fonte 15 36
Figura 27 – Resultado do código-fonte 16 37
Figura 28 – Resultado do código-fonte 16 38
Figura 29 – Resultado do código-fonte 17 39
Figura 30 – Resultado do código-fonte 18 40
Figura 31 – Resultado do código-fonte 18 41
Figura 32 – Resultado do código-fonte 19 42
Figura 33 – Resultado do código-fonte 20 43
Figura 34 – Resultado do código-fonte 22 44
Figura 35 – Resultado do código-fonte 23 46
Figura 36 – Resultado do código-fonte 24 47
Figura 37 – Resultado do código-fonte 25 48
Figura 38 – Resultado do código-fonte 27 49
Figura 39 - Resultado do código-fonte 28 50
Figura 40 – Exibição de novos arquivos criados no ambiente Google Colab.....51

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Desafios de trabalhar com dados.....23

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Operações matemáticas em NumPy 27
Código-fonte 2 – Multiplicação de matriz por escalar em NumPy 27
Código-fonte 3 – Soma de matrizes em NumPy 28
Código-fonte 4 – Criando arrays a partir de listas 28
Código-fonte 5 – Obtendo dimensões e formatos do array NumPy 28
Código-fonte 6 – Outras operações matemáticas com arrays NumPy.....29
Código-fonte 7 – Alterando tamanhos de arrays NumPy 30
Código-fonte 8 – Comparando tempo de execução de listas Python com arrays NumPy 31
Código-fonte 9 – Criando uma Series do Pandas 33
Código-fonte 10 – Criando uma Series do Pandas 33
Código-fonte 11 – Criando uma Series do Pandas indexada por letras.....34
Código-fonte 12 – Indexando uma Series do Pandas indexada por letras.....34
Código-fonte 13 – Condicionais e filtros no Pandas.....34
Código-fonte 14 – Condicionais e filtros no Pandas.....35
Código-fonte 15 – Obtendo valores do DataFrame.....36
Código-fonte 16 – Criando novas colunas em um DataFrame Pandas.....37
Código-fonte 17 – Criando novas colunas em um DataFrame Pandas.....37
Código-fonte 18 – Leitura de um arquivo de dados com Pandas.....39
Código-fonte 19 – Exibição de 5 linhas aleatórias.....40
Código-fonte 20 – Obtendo informações das variáveis do DataFrame Pamdas.....40
Código-fonte 21 – Visualizando dados faltantes com a biblioteca missingno 41
Código-fonte 22 – Obtendo análises estatísticas básicas do Pandas.....42
Código-fonte 23 – Obtendo análises estatísticas básicas do Pandas.....43
Código-fonte 24 – Limpeza de dados com Pandas.....44
Código-fonte 25 – Tratando dados categóricos ordinais.....45
Código-fonte 26 – Tratando dados categóricos nominais 46
Código-fonte 27 – Como eliminar colunas via Pandas.....47
Código-fonte 28 – Trazendo dados para a mesma escala.....48
Código-fonte 29 – Criando um DataFrame com os dados Padronizados 49
Código-fonte 30 – Criando um DataFrame com os dados Normalizados 49
Código-fonte 31 – Exportando o resultado das transformações aplicadas 50

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

MODELAGEM DE DADOS 6
1 INTRODUÇÃO 6
2 O MUNDO DOS DADOS.....7
2.1 Classificação de dados de acordo com sua fonte 8
2.2 Classificação de dados de acordo com sua granularidade 9
2.3 Classificação de dados de acordo com sua estrutura 9
2.4 Classificação de dados de acordo com seu tipo 10
2.5 Organização dos dados.....11
3 O UNIVERSO DA CIÊNCIA DE DADOS.....12
3.1 Trabalhando com variáveis 15
3.1.1 Técnicas de amostragem para conjuntos de treino/validação/teste 17
3.1.2 Pré-processamento de dados 18
3.1.3 Engenharia de features (*feature engineering*) 19
4 DESAFIOS DO TRABALHO COM DADOS.....20
4.1 Overfitting 23
4.2 Underfitting 24
4.3 Modelo ideal 25
5 CIÊNCIA DE DADOS EM PYTHON: BIBLIOTECAS NUMPY E PANDAS.....26
5.1 NumPy.....27
5.2 Pandas 32
5.3 Explorando e limpando dados com Pandas 38
5.4 Engenharia de features com Pandas e Scikit-learn.....45
6 CONCLUSÃO.....51
REFERÊNCIAS.....52
GLOSSÁRIO 53

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

MODELAGEM DE DADOS

1 INTRODUÇÃO

Bem-vindo(a) à disciplina "**Deep Learning e AI**"! Essa disciplina complementa sua formação como Cientista de Dados, pois introduzirá conceitos e ferramentas importantes sobre Inteligência Artificial e Redes Neurais Artificiais. Você verá que essa disciplina guarda relações estreitas com "Aprendizado de Máquina", pois as Redes Neurais Artificiais são uma família de algoritmos de Machine Learning. No entanto, por ser um algoritmo mais complexo, e que evoluiu muito nos últimos anos, as Redes Neurais Artificiais merecem um destaque especial.

Basta uma simples consulta na Internet e você encontrará uma série de notícias que indicam a **importância da Inteligência Artificial, do Aprendizado de Máquina e da Ciência de Dados no mundo atual**. Dizemos que estamos vivendo na era "*data-driven*", quando as pessoas e empresas passaram a tomar decisões estratégicas cada vez mais baseadas em dados e evidências. Mas vamos começar com uma provocação. Sabemos que:

- **As pessoas sempre tomaram decisões**: a história é uma série de decisões tomadas ao longo do tempo;
- **Dados sempre foram coletados**: eles podem estar pintados na parede da caverna, entalhados na madeira, anotados em papel ou até já nascerem no mundo digital (isso é uma evolução tecnológica para um ato muito antigo);
- **Toda decisão é baseada em informação**: algumas decisões podem ser tomadas sobre informações falsas, mas sempre que ponderamos sobre algo, fazemos baseado nas informações que temos naquele momento.

Então pergunto: se os três itens acima são verdadeiros, o que há de novo na chamada era "*data-driven*"?

A resposta é que, atualmente, temos uma **sistematização** dos processos de coleta, armazenamento e processamento de dados e, sobretudo, na **geração de**

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

insights e tomada de decisão. Hoje em dia, sistemas computacionais influenciam o que vamos comprar, quais entretenimentos podemos ter e até mesmo com quem vamos interagir em redes sociais. Assim, é natural delegar ao menos parte das decisões para as máquinas realizarem em qualquer setor produtivo.

Essa revolução foi impulsionada por três fatores:

- **Aumento da quantidade de dados** (importante para melhorar performance dos métodos de Inteligência Artificial);
- **Avanços em capacidade computacional** (modelos de Machine Learning exigem muita memória);
- **Criação de novos algoritmos para problemas mais complexos** (por exemplo, o surgimento de Deep Learning, que representa o estado da arte para processamento massivo de imagens, textos, áudios e vídeos).

Essa disciplina proverá os recursos necessários para você compreender os conceitos de Deep Learning, conhecer suas aplicações e saber trabalhar com tipos de dados mais desafiadores. Aproveitaremos a oportunidade para reforçar conceitos de Aprendizado de Máquina, igualmente essenciais para trabalharmos de maneira correta com Deep Learning.

Nossa jornada começará na **Modelagem de Dados**. Vamos entender os conceitos sobre dados, seus tipos e como trabalhar adequadamente com eles para gerarmos insights úteis e relevantes. Esse passo é a base para permitir trabalhar com algoritmos mais sofisticados, como Redes Neurais Artificiais, e, posteriormente, mergulharmos na sua evolução: o aprendizado profundo (Deep Learning).

2 O MUNDO DOS DADOS

Os dados são o combustível essencial para trabalharmos com Ciência de Dados e, como veremos, as formas mais modernas de Inteligência Artificial. Aqui, vamos nos aprofundar nas características dos dados e suas formas de armazenamento.

A primeira pergunta é: o que é o **dado**? Ao pensar nessa questão, muita gente pode relacionar “dado” com “informação”, mas para sermos mais precisos, dizemos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

que o dado é “**um valor coletado e armazenado**”. O dado tem o *potencial* de relevar uma informação quando está relacionado com uma **variável**.

Vamos exemplificar: podemos perguntar as idades dos alunos dessa disciplina, e obtermos valores como 17, 18, 19, 20, 21, 22, 25 (anos), entre outros. Cada número apresentado é um *valor*, que está relacionado com a *variável* idade. Se pegarmos o número “23” sozinho, o que ele representa? O valor de uma compra? A temperatura do dia? O número de uma casa? Se soubermos que “23” é a idade de um aluno, aí sim estamos diante da *informação*.

Portanto, quanto mais variáveis nós tivermos sobre um contexto, maior é a quantidade de informações que teremos para tomar decisões. Podemos entender os dados de acordo com várias dimensões, como veremos a seguir.

2.1 Classificação de dados de acordo com sua fonte

De onde os dados provêm? Podemos estar diante de três tipos de fontes de dados:

- **Dados primários**: quando detemos a fonte de geração daqueles valores. Geralmente são exemplos de dados privados de uma empresa, por exemplo, sua receita, o número de peças produzidas, o lucro de cada produto etc. Aqui, a fonte de informação é própria, e geralmente não é compartilhada de forma bruta;
- **Dados secundários**: são dados primários de outra empresa/companhia, mas que foram cedidos para nossa análise. Por exemplo, a empresa A compõe o preço de venda do seu produto mediante uma série de fatores, incluindo os componentes isolados adquiridos das empresas B e C. As empresas B e C possuem seus dados primários de “valor de venda”, mas esses são conhecidos pela empresa A para compor o valor final do seu produto;
- **Dados terciários**: são dados de uma empresa/companhia qualquer que já são disponibilizados de forma pública, ou seja, qualquer um pode ter acesso àquela informação. Nosso foco não está em quem produziu o dado, mas sim no dado em si;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2 Classificação de dados de acordo com sua granularidade

Em qual nível os dados foram cedidos?

- **Dados agregados**: os dados originais foram previamente consolidados ou agrupados de acordo com algum critério. Por exemplo, qual é a idade média dos alunos que cursam “Deep Learning & AI”? Qual foi a nota final média desses alunos nessa disciplina? De forma geral, os dados agregados não permitem identificar quem é quem, pois já foram tratados antes de serem disponibilizados;
- **Dados desagregados**: aqui, os dados estão em sua forma mais bruta, da forma como foram coletados. Por exemplo, informações de cada aluno que cursou “Deep Learning & AI”, nas quais posso identificar quem é quem, sua idade e sua nota final.

2.3 Classificação de dados de acordo com sua estrutura

De qual forma os dados são apresentados?

- **Dados estruturados**: nesse formato, os dados são apresentados no formato de tabela, com linhas e colunas, sendo que, em cada coluna, temos uma variável e, em cada linha, uma entrada daquelas informações. Por exemplo, no cadastro de um cliente podemos ter as variáveis nome, sobrenome, e-mail e celular. Aqui, temos 4 variáveis (colunas) e cada cliente é uma entrada na base de dados (linha);
- **Dados semiestruturados**: existe uma estrutura parcial, não tão fixa como uma tabela, mas há uma receita de como estruturar a informação, geralmente por meio de tags ou chave-valor. Exemplos mais comuns são páginas HTML e arquivos JSON;
- **Dados não-estruturados**: referem-se a informações que não estão organizadas em um formato predefinido ou rígido, como tabelas, tags ou pares de chave-valor. Esses dados não seguem um esquema fixo, tornando-os mais desafiadores de serem interpretados e analisados por meio de métodos tradicionais. Exemplos incluem imagens, vídeos,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

textos, áudios e postagens em redes sociais (veja Figura 1 – Exemplos de tipos de estruturas de dados).

Nome	CPF	Endereço
Marcela Freitas	11111	Rua A, nº 1
João Augusto	22222	Rua B, nº 2
Pablo Silva	33333	Rua C, nº 3
André Mendes	44444	Rua D, nº 4
Juliana Freitas	55555	Rua E, nº 5

Dado estruturado (tabela)


```
{
    bairro: "Farolândia",
    cidade: "Aracaju",
    cep: "49032490",
    logradouro: "Avenida Murilo Dantas",
  - estado_info: {
        area_km2: "21.918,493",
        codigo_ibge: "28",
        nome: "Sergipe"
    },
  - cidade_info: {
        area_km2: "181,857",
        codigo_ibge: "2800308"
    },
    estado: "SE"
}
```


Dado semiestruturado (JSON)

Dado não-estruturado (imagem)

Figura 1 – Exemplos de tipos de estruturas de dados
Fonte: autor

2.4 Classificação de dados de acordo com seu tipo

Que tipo de informação os dados trazem?

- **Dados numéricos**: esse tipo de variável carrega uma informação *quantitativa*, ou seja, que expressa uma quantidade. Podemos ter *valores discretos* (números inteiros – exemplos: quantidade de filhos, número de banheiros em uma residência) ou *valores contínuos* (números reais – exemplos: salário, altura);
- **Dados categóricos**: esse tipo de variável carrega uma informação *qualitativa*, ou seja, que expressa uma característica. Podemos ter *valores nominais* (quando não há uma relação de ordem entre as possibilidades de resposta – exemplos: sexo biológico, profissão) ou *valores ordinais* (quando há uma relação de ordem entre as possibilidades de resposta – exemplos: posição em um ranking, nível de escolaridade).

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

OBSERVAÇÃO:

Neste curso, trabalharemos majoritariamente com *dados terciários* e *desagregados*. A primeira parte, focada em Redes Neurais Artificiais, focará em dados *estruturados* (que podem ser *numéricos* ou *categóricos*); quando entrarmos em Deep Learning, trabalharemos com dados *não-estruturados*.

2.5 Organização dos dados

Os dados são organizados e armazenados em **bancos de dados**, que são conjuntos organizado e estruturados de informações relacionadas, geralmente armazenadas eletronicamente em um Sistema de Gerenciamento de Banco de Dados (SGBD). Essas informações são organizadas em tabelas, onde cada tabela representa uma entidade específica e as relações entre elas. Os bancos de dados permitem armazenar, recuperar, gerenciar e manipular dados de forma eficiente, fornecendo uma base sólida para aplicativos e sistemas que requerem acesso rápido e confiável às informações. São amplamente utilizados em várias áreas, como negócios, ciência, governo e tecnologia.

Também existem bancos de dados especialmente projetados para armazenamento e busca de dados não-estruturados. A discussão de banco de dados para dados estruturados ou não-estruturados está fora do escopo dessa disciplina. O que devemos tratar aqui é que a “evolução” dos bancos de dados culminaram no “Big Data”.

O Big Data é um conjunto massivo de dados representado por 5 características essenciais (veja Figura 2):

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

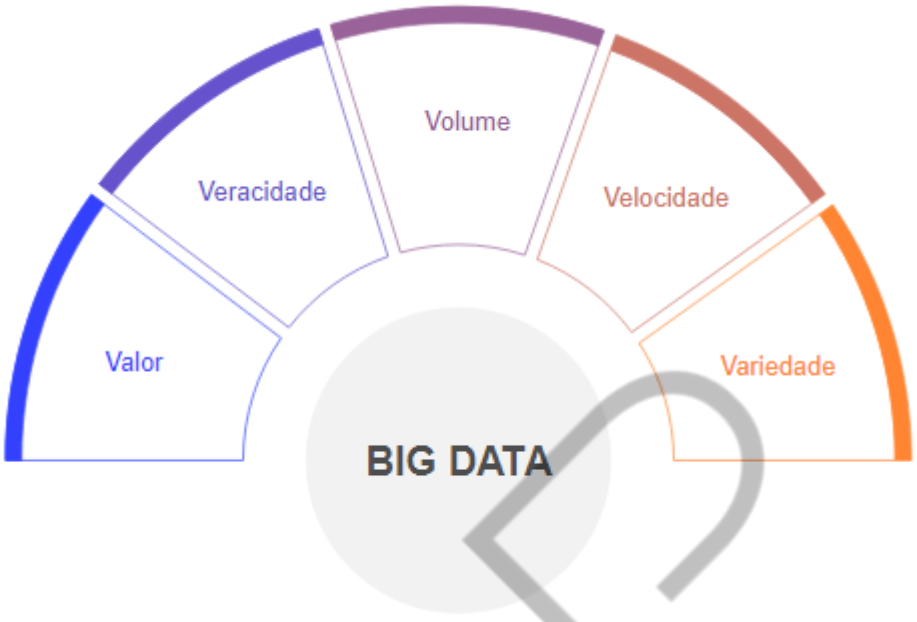

Figura 2 – Os 5 V's do Big Data
Fonte: autor

Tais repositórios de Big Data são caracterizados pelo seu **volume** (quantidade massiva de dados), sua **variedade** (podendo conter dados estruturados, semiestruturados e não-estruturados), produzidos e armazenados com alta **velocidade** e que possuem **valor** para obtenção de insights e que devem ser armazenados sob o princípio da **veracidade** (ainda que todo dado possua problema de qualidade, devemos armazenar dados mais fidedignos possíveis).

No contexto de Ciência de Dados, o Big Data passou a ser a "principal fonte" de dados para as análises, representando com conjunto massivo e amplo de dados. O conceito de Big Data é ainda mais importante para Deep Learning, quando a quantidade de dados para construir os modelos de Inteligência Artificial é ainda maior.

3 O UNIVERSO DA CIÊNCIA DE DADOS

A pergunta central dessa seção é: **como desenvolver soluções baseadas em Ciência de Dados**? Para responder essa questão, vamos nos basear no ciclo de vida das soluções de Ciência de Dados da Figura 3 abaixo:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 3 – Ciclo de vida de soluções baseadas em Ciência de Dados
Fonte: https://medium.com/@vanshikagoel/data-science-life-cycle-in-5-steps-3fe720b2a2fe

As etapas do ciclo são:

1. **Business understanding (entendimento do negócio)**: nessa etapa inicial, devemos estabelecer o objetivo da nossa análise de dados e o tipo de resultado que queremos construir: será um relatório, um painel interativo, um modelo preditivo?;
2. **Data mining (obtenção dos dados)**: agora que sabemos nosso objetivo, vamos buscar os dados que apoiam nossa análise. Essa etapa é a seleção dos dados que comporão nossa base de dados a ser trabalhada;
3. **Data cleaning (limpeza dos dados)**: é natural que os dados venham com problemas, seja questões de falta de informações ou valores imprecisos. Essa etapa trata os dados realizando uma limpeza para remover valores estranhos ou diferentes do que era o esperado;
4. **Data exploration (exploração dos dados)**: a etapa de exploração de dados é fundamental para aprofundar nosso conhecimento sobre o tema sendo investigado. Aqui, encontramos padrões mais simples, correlações entre os dados, observamos valores médios e medianos. A

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

aplicação de estatística básica nessa etapa e construção de alguns gráficos ampliam nosso conhecimento sobre o tema sendo explorado;

5. **Feature engineering (engenharia de features)**: essa etapa está mais relacionada com modelos preditivos, quando padronizamos variáveis e/ou eliminamos variáveis desnecessárias ou pouco informativas e/ou criamos novas variáveis a partir das já existentes. Tudo isso para termos um conjunto mais refinado de variáveis para a máquina aprender a fazer predições;
6. **Predictive modeling (modelagem preditiva)**: nem toda análise de dados exige a construção de modelos preditivos, mas quando o entendimento de negócio exige esse tipo de entrega, é nessa etapa que de fato treinamos e avaliamos o modelo. Nas soluções de Deep Learning, a construção da modelagem preditiva é o ponto central a ser feito. Falaremos mais sobre essa etapa adiante;
7. **Data visualization (visualização de dados/comunicação)**: seja após a construção do modelo preditivo ou após a construção de um relatório baseado em dados, a última etapa do ciclo é a demonstração da solução para quem a encomendou, verificando o atendimento dos requisitos, oportunidades de melhoria ou simplesmente para aceitar a entrega como está, pois ela já possui potencial de impacto positivo no negócio.

Vale destacar que o ciclo de vida das soluções baseadas em Ciência de Dados é iterativo. A partir de uma etapa, podemos ir para a próxima ou voltar à anterior. Por exemplo, após a limpeza dos dados, descobrimos que não temos informações confiáveis suficientes para construir um modelo preditivo, portanto, devemos voltar à etapa de coleta de dados. Além disso, é comum que as soluções passem por N ciclos dessas etapas até estarem maduras o suficiente para sua utilização.

Essa iteração durante o desenvolvimento é muito positiva, pois aproxima os demandantes das soluções (geralmente pessoas com amplo conhecimento de *negócio*, mas pouco conhecimento *técnico*), da equipe de ciência de dados (geralmente pessoas com amplo conhecimento *técnico*, mas pouco conhecimento sobre o *domínio* de onde os dados provêm).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

As próximas seções deste capítulo trarão os **aspectos técnicos** de cada etapa. Vamos assumir que o entendimento de negócio já foi feito e que os dados já foram coletados, portanto, vamos nos atentar às etapas de *limpeza de dados*, *exploração de dados* e *engenharia de features*. Primeiramente, vamos compreender as motivações e passos de cada etapa; a parte prática deste capítulo será a implementação em Python de cada passo listado.

Nessa disciplina, nosso alvo de modelagem será sempre modelos preditivos, portanto, focaremos nessa forma de modelagem. Análises descritivas, diagnósticas e prescritivas também são muito importantes em Ciência de Dados, e serão tratadas em outras disciplinas.

3.1 Trabalhando com variáveis

Será que basta ter os dados em mãos para trabalhamos com Ciência de Dados? Não! Antes de mais nada, precisamos aprender *como* usar as bases de dados, especialmente para criarmos modelos preditivos.

Precisamos separar nossos dados em 2 conjuntos disjuntos: o treinamento e o teste. Eles são disjuntos porque os exemplos que estão no treinamento não podem estar no teste e vice-versa (pois isso comprometeria a avaliação da efetividade da modelagem preditiva). Normalmente, essa separação é feita de forma aleatória, separando 80% dos dados para treinar e 20% para teste, mas outras combinações podem ser aplicadas (como 70%/30%). Veja a Figura 4.

Figura 4 – Separação de dados em treino e teste
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Os dados do subconjunto de **treino** são utilizados para o algoritmo de Machine Learning ou Deep Learning investigar padrões, identificar correlações e aprender a mapear relações nos dados de acordo com a tarefa que está sendo aprendida. Por exemplo, a máquina pode analisar diversas imagens de lesões de pele para aprender a reconhecer quais são lesões malignas e quais são lesões comuns. Os dados de treino são acessados quantas vezes e de qual forma a máquina precisar durante o treinamento do algoritmo.

Já os dados de **teste** são utilizados para avaliar o poder preditivo do algoritmo de Machine Learning ou Deep Learning naquela tarefa em questão. Por exemplo, após treinar a máquina a reconhecer melanomas (um tipo de câncer de pele) em lesões de pele, queremos investigar qual a capacidade preditiva real desse modelo, fazendo-o gerar predições para dados que nunca foram vistos anteriormente (o subconjunto de teste – percebeu por que devemos separar completamente os subconjuntos de treino/teste?).

Você pode estar se perguntando: “mas como a máquina sabe, em tempo de treinamento, que ela está aprendendo algo de verdade?”. Muito bem! Além da divisão entre treino e teste, é comum subdividir o conjunto de treino em outros 2: o **treino** propriamente dito e a **validação** (veja Figura 5).

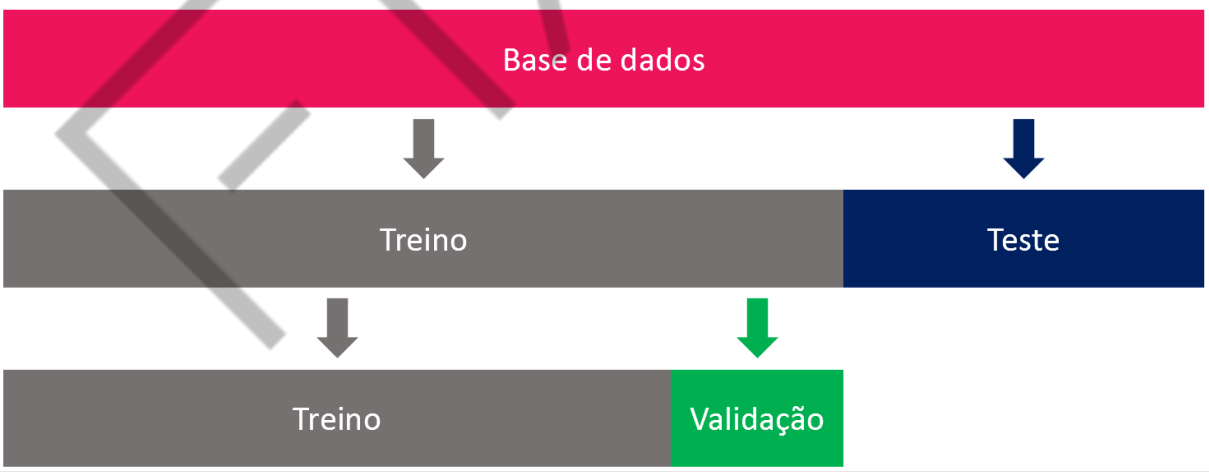

Figura 5 – Separação de dados em treino, validação e teste
Fonte: autor

A proporção mais utilizada é 60% da base de dados para treino, 20% para validação e 20% para teste. O conceito de separação disjunta continua válido nessa

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

nova subdivisão: o exemplo que está em um dos conjuntos NÃO pode estar presente nos demais.

A importância do subconjunto de validação é a máquina ter uma amostra separada de dados para “brincar” de fazer testes em tempo de treinamento para identificar se os padrões aprendidos realmente resolvem a tarefa ou não está havendo aprendizado real. Na prática, o treinamento encerra quando não há mais como melhorar os acertos de predição no subconjunto de validação. Na prática, ainda, o próprio algoritmo de Machine Learning ou Deep Learning subdivide o treino em treino e validação, cabendo a nós apesar lembrar de separar treino e teste.

3.1.1 TÉCNICAS DE AMOSTRAGEM PARA CONJUNTOS DE TREINO/VALIDAÇÃO/TESTE

Existem diversas técnicas de amostragem para conjuntos de treino, validação e teste:

- **Amostragem Aleatória Simples**: seleção aleatória de exemplos do conjunto de dados, garantindo que cada exemplo tenha a mesma probabilidade de ser escolhido;
- **Amostragem Estratificada**: garante que a proporção de cada classe no conjunto de treinamento, validação e teste seja representativa da proporção geral do conjunto de dados;
- **Amostragem Cruzada (Cross-Validation)**: divide o conjunto de dados em k partições (folds) e executa o treinamento e validação k vezes, usando cada partição como conjunto de teste em uma iteração diferente;
- **Amostragem por Bootstrap**: cria múltiplas amostras de treinamento aleatórias, com reposição, a partir do conjunto de dados original, para treinar modelos em diferentes conjuntos de treinamento;
- **Amostragem por Minibatch**: divide o conjunto de treinamento em pequenos lotes (minibatches) para treinar o modelo de forma mais

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

eficiente, especialmente quando o conjunto de dados é grande demais para caber na memória;

- **Amostragem por Tempo**: em séries temporais, usa dados passados para treinamento e dados futuros para teste, simulando a previsão do comportamento do modelo em situações reais.

Essas técnicas de amostragem são cruciais para garantir que os modelos de Deep Learning e Machine Learning sejam bem treinados e generalizem bem para novos dados. Cada uma delas tem suas vantagens e pode ser usada de acordo com a natureza do problema e o tamanho do conjunto de dados disponível. Não se preocupe: vamos detalhar cada uma delas oportunamente nessa disciplina.

3.1.2 PRÉ-PROCESSAMENTO DE DADOS

Sabendo que toda base de dados possui problemas de qualidade, a etapa de pré-processamento de dados engloba limpeza dos dados e também exploração da base, para o cientista de dados conhecer cada vez mais do desafio a ser modelado.

É comum que a exploração da base siga junto com sua limpeza, pois conforme nós aprendemos mais sobre os dados (vamos explorando aquela base), mas nos deparamos com problemas a serem ajustados. E conforme os problemas vão sendo corrigidos, mais vamos conhecendo sobre o que cada variável tem a nos dizer.

A **exploração da base** costuma ser embasada numa série de perguntas que colocamos sobre os dados, tais como:

- Qual é o tamanho dessa base de dados? Quantos exemplos (linhas) e variáveis (colunas) nós temos?;
- Quais variáveis são numéricas e quais são categóricas? Quais são discretas e quais são contínuas? Quais são nominais e quais são ordinais?;
- Existem outliers nos dados?;
- Como estão distribuídas as classes em um problema de classificação?;
- Há correlações significativas entre variáveis?;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Quais são as estatísticas descritivas básicas das variáveis numéricas?;
- Quais são os tipos de dados e quantidades de valores nulos?;
- Quantas ocorrências de cada valor único das variáveis categóricas?;
- Entre outras questões específicas de cada problema!

Por outro lado, a **limpeza da base** consiste em identificar problemas descobertos diretamente na exploração da base, tais como:

- Identificação e remoção de valores nulos;
- Preenchimento de valores nulos evitando a remoção;
- Identificação e remoção de outliers.

3.1.3 ENGENHARIA DE FEATURES (*FEATURE ENGINEERING*)

A engenharia de features consiste em uma série de transformações de dados mais elaboradas e que muitas vezes exige conhecimento sobre o domínio de geração dos dados. São exemplos de tais transformações:

- **Seleção de features**: dadas as variáveis existentes, quais serão levadas adiante nas modelagens e quais serão excluídas, ou porque não interessam ao negócio ou porque geram implicações éticas;
- **Criação de features**: dadas as variáveis existentes, quais outras podem ser criadas a partir delas? E essa combinação de features faz sentido? Por exemplo: calcular IMC a partir de altura e peso; calcular idade a partir da data de nascimento; a partir da data, descobrir se é um dia útil ou não;
- **Redução de dimensionalidade**: cada variável representa uma dimensão a ser avaliada. Se tivermos 30 variáveis em um dataset, estaremos lidando com 30 dimensões. Quanto mais dimensões nos dados, mais complexa fica sua modelagem preditiva. Portanto, uma etapa a ser considerada na limpeza/engenharia de recursos é a eliminação de variáveis de baixo valor preditivo ou baixa relação com o problema analisado. Isso diminui a dimensionalidade do problema.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Outras técnicas mais sofisticadas, como o PCA (*Principal Component Analysis*), podem ser exploradas na redução de dimensionalidade;

- **Normalização de dados**: é comum termos em uma mesma base variáveis expressas em escalas diferentes. Por exemplo: a diferença entre idades pode ser expressa em unidades ou dezenas, mas a diferença entre salários pode ser expressa em milhares ou dezenas de milhares. No universo de modelos preditivos, é importante levar todas as variáveis para a mesma escala, para que o modelo aprenda as relações entre elas no mesmo pé de igualdade. Sem isso, uma diferença expressa em alta dimensão pode passar a impressão falsa de que aquela variável é mais relevante que as demais para uma predição qualquer.

OBSERVAÇÃO:

A parte prática dessa unidade mostrará como a exploração, limpeza e engenharia de features são feitas utilizando bibliotecas de Python.

4 DESAFIOS DO TRABALHO COM DADOS

Trabalhar com dados é fascinante! Quanto mais nos aprofundamos sobre uma base de dados, mais aprendemos sobre aquele domínio e mais nos apaixonamos pelo tema. No entanto, nem tudo são flores: existem diversos desafios para se trabalhar com dados. Observe a Figura 6 abaixo:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave_points_mean	symmetry_mean	fractal_dimension_mean
B	13,54	14,36	87,46	566,3	0,09779	0,08129	0,06664	0,04781	0,1885	0,05766
B	13,08	15,71	85,63	520	0,1075	127	0,04568	0,0311	0,1967	0,06811
B	8196	16,84	51,71	201,9	86	0,05943	0,01588	0,005917	0,1769	0,06503
B	6981	13,43	43,79	143,5	117	0,07568	0	0	193	0,07818
B	12,18	20,52	77,22	458,7	0,08013	0,04038	0,02383	0,0177	0,1739	0,05677
M	25,22	24,91	171,5	1878	0,1063	0,2665	0,3339	0,1845	0,1829	0,06782
M	19,1	26,29	129,1	1132	0,1215	0,1791	0,1937	0,1469	0,1634	0,07224
M	18,46	18,52	121,1	1075	0,09874	0,1053	0,1335	0,08795	0,2132	0,06022
M	14,48	21,46	94,25	648,2	0,09444	0,09947	0,1204	0,04938	0,2075	0,05636
M	19,02	24,59	122	1076	0,09029	0,1206	0,1468	0,08271	0,1953	0,05629

Figura 6 – Fragmento de uma base de dados real
Fonte: autor

A Figura 6 ilustra um fragmento de uma base de dados real. Certamente existem mais linhas nessa planilha, mas esse pequeno fragmento já nos instiga a descobrirmos os principais desafios. Sabemos que cada linha é um exemplo de dado, e cada coluna é uma variável, mas:

1. **Que dados são esses**: o primeiro desafio é do próprio entendimento de negócio. De qual domínio eles derivam? O que significa, por exemplo, “área média”?;

- Para abrir o jogo, estes são linhas de uma base de dados disponível no Kaggle com informações sobre câncer de mama. Cada linha contém dados numéricos que caracterizam nódulos do seio, a partir da análise de imagens médicas de mamografias.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

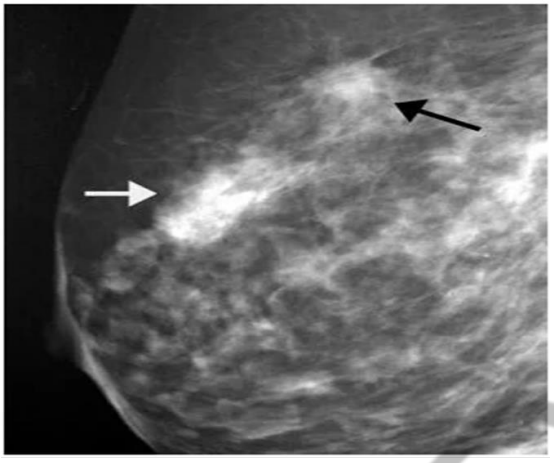

Figura 7 – Exemplo de mamografia com destaque para dois nódulos
Fonte: https://www.canceroz.com/2017/02/breast-ultrasound-cancer-vs-benign.html

Ou seja, a partir da mamografia, pegamos a imagem médica, identificamos os nódulos e para cada um, descremos seu raio, sua textura em escala de cinza, seu perímetro, área, entre tantas outras variáveis que são expressas de forma numérica. A letra “B” significa um diagnóstico benigno e a letra “M”, um diagnóstico maligno. Agora entendemos a origem dos dados e temos uma melhor ideia do que representam cada variável apresentada;

2. **Conhecer a distribuição dos dados**: podemos nos perguntar como está a distribuição desses dados, quais são os valores mínimo e máximo de cada variável, se os dados estão desbalanceados (proporção desigual entre exemplos “B” e “M”), se existem dados faltantes, se existem outliers etc.;
3. **Preparar os dados para uso**: antes de colocar o algoritmo para treinar, precisamos preparar a base, que é a etapa de limpeza de dados e engenharia de features. Isso é: será que todas as variáveis são necessárias? O que fazer com dados faltantes? Os campos com “zero” são dados reais ou valores faltantes? Como reconhecer e eliminar outliers?

Quando trabalhamos com dados, ainda temos outros desafios, que podem ser causados por **dados ruins** ou **algoritmos ruins**:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Problemas causados por dados ruins	Problemas causados por algoritmos ruins
• Dados insuficientes para treinar os modelos; • Dados com baixa qualidade; • Features irrelevantes.	• Overfitting do modelo; • Underfitting do modelo.

Quadro 1 – Desafios de trabalhar com dados
Fonte: autor

Quando o problema está no domínio dos dados, devemos trabalhar na base de dados para resolvê-lo. Quando nossa base de dados é pequena, não há o que fazer, precisamos coletar novos exemplos. Quando nossos dados possuem baixa qualidade, devemos investir ainda mais tempo na limpeza de dados. Quando as features são irrelevantes, podemos ignorar algumas delas ou tentar criar features mais poderosas. Por exemplo: podemos criar o IMC a partir do peso e altura de uma pessoa.

Quando o problema está no algoritmo, devemos trocá-lo ou refinar o processo de treinamento do modelo de Machine Learning ou Deep Learning. Vamos reconhecer esses problemas agora.

4.1 Overfitting

O overfitting, também conhecido como sobreajuste, acontece quando o algoritmo preditivo é complexo demais para diferenciar as variações dos dados. No caso, o modelo se adapta excessivamente aos dados de treinamento. Isso acontece quando o modelo aprende detalhes e ruídos específicos dos dados de treinamento em vez de aprender padrões gerais que possam ser aplicados a novos dados.

Em outras palavras, o modelo memoriza os dados de treinamento em vez de aprender a generalizar e capturar a relação subjacente entre os recursos e a variável alvo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 8 – Exemplos lúdicos de overfitting
Fonte: https://www.google.com/search?q=overfitting+meme&tbm=isch&ved=2ahUKEwiIu4fy1bSAAxX9NLkGHWYUBr0Q2-cCegQIABAA&oq=overfitting+meme&gs_lcp=CgNpbWcQAzIHCAAQExCABDoECCMQJzoFCAAQgAQ6BggAEAcQHjoECAAQHjoICAAQBRAeEBM6CAgAEAgQHhATUMkFWN8JYPcLaABwAHgAgAF3iAGDBZIBAzEuNZgBAKABAaoBC2d3cy13aXotaW1nwAEB&sclient=img&ei=JmvFZIiQNf3p5OUP5qiY6As&bih=609&biw=1280&rlz=1C1GCEU_pt-BRBR1048BR1048#imgrc=JVm4YvaYKzRBlM

Considere os exemplos da Figura 8. Eles representam, de forma lúdica, o que é o sobreajuste. No primeiro caso, queremos construir uma casinha de cachorro, e o resultado é uma casinha de cachorro que só serve para AQUELE cachorro NAQUELA posição. Qualquer outra tentativa de usar essa casinha em outra situação ela seria complementa inútil. No segundo exemplo, considere a cama apresentada. Para alguém que somente dorme EXATAMENTE nessa posição, ela é ótima, pois não falta nada nem sobra. Para qualquer outra pessoa, essa cama seria péssima! Ou seja, é como se o modelo tivesse “decorado” o que é esperado para o dado que foi apresentado durante sua construção (decorou os dados do treino).

Overfitting pode ocorrer quando o modelo é muito complexo em relação ao tamanho do conjunto de treinamento ou quando o conjunto de treinamento é insuficiente. Para combater o overfitting, são usadas técnicas de regularização, como dropout, L1 e L2, bem como a validação cruzada para avaliar o desempenho do modelo em dados não vistos. Veremos essas técnicas adiante!

4.2 Underfitting

Underfitting (subajuste) é outro fenômeno indesejável que pode ocorrer em modelos de aprendizado de máquina, incluindo redes neurais em Deep Learning. Ao

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

contrário do overfitting, o underfitting ocorre quando o modelo não é capaz de capturar adequadamente os padrões e a relação subjacente nos dados de treinamento (imagine tentar matar um dragão com um mata-moscas).

Quando um modelo sofre de underfitting, ele é muito simples para lidar com a complexidade dos dados de treinamento. Como resultado, o modelo não consegue aprender nem representar adequadamente os padrões e relacionamentos nos dados, levando a uma baixa capacidade de generalização. O desempenho do modelo no conjunto de treinamento também pode ser insatisfatório.

As principais causas do underfitting incluem o uso de um modelo muito simples, com poucos parâmetros, ou o treinamento insuficiente do modelo. Além disso, um conjunto de dados pequeno ou de baixa qualidade pode contribuir para o subajuste.

4.3 Modelo ideal

Então o que é um modelo ideal? É importante encontrar um equilíbrio entre um modelo que é suficientemente complexo para aprender os padrões relevantes e não tão complexo a ponto de memorizar os dados de treinamento.

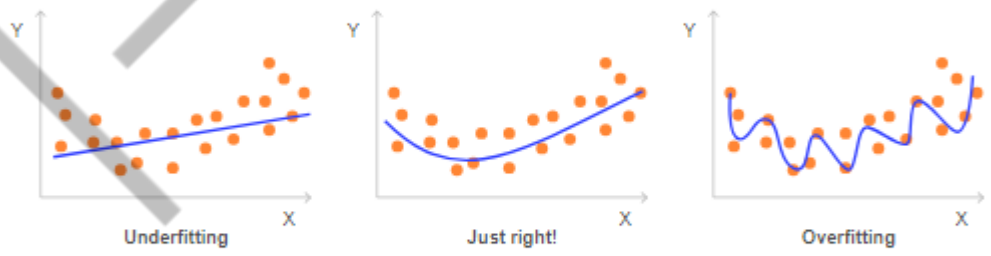

Figura 9 – Definição de modelo ideal
Fonte: https://www.analyticsvidhya.com/blog/2021/06/complete-guide-to-prevent-overfitting-in-neural-networks-part-1/

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A Figura 9 ilustra o modelo ideal (central) ao lado de exemplos de underfitting e overfitting. Em todos os casos, os pontos em vermelho indicam os dados de treinamento e a curva em azul indica o modelo preditivo que ilustra o comportamento de tais dados.

No overfitting, o modelo em azul é demasiadamente complexo, representando EXATAMENTE todos os dados, sem captar suas tendência e características gerais. Portanto, o modelo falha na generalização do aprendizado, pois não houve aprendizado: apenas decorou os exemplos.

Já no undefitting, o modelo em azul é demasiadamente simples, sendo incapaz de reconhecer as variações nos dados. Aqui, o modelo forçou uma representação linear da distribuição dos dados, quando naturalmente a distribuição é um pouco mais sofisticada.

Por fim, o modelo ideal (figura central) é perfeito: não é nem simples demais, nem complexo demais, pois modela bem o comportamento dos dados, mas possui certa tolerância a novos dados.

5 CIÊNCIA DE DADOS EM PYTHON: BIBLIOTECAS NUMPY E PANDAS

Agora que passamos pelos principais conceitos da nossa jornada de trabalho com dados, rumo à criação de modelos preditivos, está na hora de conhecermos as principais bibliotecas em Python que implementam as etapas listadas acima. Começaremos vendo duas bibliotecas muito importantes, o NumPy e o Pandas, e logo começaremos a entender a exploração e limpeza de dados em Python.

Nessa primeira prática, vamos conhecer os comandos centrais dessas bibliotecas. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

DICA: Veja o material HOW TO para acessar o ambiente do Colab.

DICA: Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 1), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

5.1 NumPy

O NumPy é uma biblioteca fundamental em Python para computação numérica e manipulação de arrays multidimensionais. Ele oferece uma ampla gama de funções matemáticas eficientes e operações de álgebra linear, tornando-se essencial em ciência de dados e, especialmente, em Deep Learning.

Sua eficiência na manipulação de grandes conjuntos de dados e a implementação de operações vetorizadas aceleram os cálculos, proporcionando um ambiente otimizado para o treinamento de modelos complexos. A capacidade do NumPy de lidar com arrays n-dimensionais é crucial para representar dados em tensores, a estrutura fundamental em redes neurais, tornando-o uma peça-chave no desenvolvimento eficaz de algoritmos de aprendizado profundo.

É possível escrever operações matemáticas aplicadas ao array do NumPy:

```
import numpy as np
data = np.random.randn(2, 3) # linhas, colunas
data
```

Código-fonte 1 – Operações matemáticas em NumPy
Fonte: Elaborado pelo autor (2023)

O comando acima produz uma saída similar à ilustrada na Figura 10. Provavelmente, sua saída será diferente, pois o código está importando a biblioteca, e gerando uma variável "`data`" que recebe uma matriz de 2 linhas e 3 colunas preenchida com valores aleatórios!

```
array([[ 0.58508518,  1.36543059, -0.14039288],
       [ 0.19296299, -0.61560359, -0.65576129]])
```

Figura 10 – Resultado do código-fonte 1
Fonte: Elaborado pelo autor (2023)

Em outras células, podemos ainda multiplicar tal matriz por 10:

```
data * 10
```

Código-fonte 2 – Multiplicação de matriz por escalar em NumPy
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
array([[ 5.8508518 , 13.65430589, -1.40392882],
       [ 1.92962988, -6.15603591, -6.55761292]])
```

Figura 11 – Resultado do código-fonte 2
Fonte: Elaborado pelo autor (2023)

Ou até mesmo somar uma matriz com outra (desde que respeitada as dimensões) de forma tão simples como somar duas variáveis:

```
data + data
```

Código-fonte 3 – Soma de matrizes em NumPy
Fonte: Elaborado pelo autor (2023)

```
array([[ 1.17017036,  2.73086118, -0.28078576],
       [ 0.38592598, -1.23120718, -1.31152258]])
```

Figura 12 – Resultado do código-fonte 3
Fonte: Elaborado pelo autor (2023)

Podemos também criar um array a partir de uma lista. Note que o numpy leva todos os números para a representação de float.

```
lista = [6, 6.75, 8, 0, 1]
arr1 = np.array( lista )
arr1
```

Código-fonte 4 – Criando arrays a partir de listas
Fonte: Elaborado pelo autor (2023)

```
array([6.  , 6.75, 8.  , 0.  , 1.  ])
```

Figura 13 – Resultado do código-fonte 4
Fonte: Elaborado pelo autor (2023)

O NumPy possui alguns atributos e métodos que podemos explorar para obtermos facilmente informações dos dados armazenados na memória. No exemplo abaixo, estamos obtendo o número de dimensões e o formato do array.

```
print(arr1.ndim)  #número de dimensões
print(arr1.shape) #formato
```

Código-fonte 5 – Obtendo dimensões e formatos do array NumPy
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
1
(5,)
```

Figura 14 – Resultado do código-fonte 5
Fonte: Elaborado pelo autor (2023)

Não é apenas a soma de matrizes que é facilitada com o NumPy. Desde que os arrays multidimensionais tenham dimensões compatíveis, podemos realizar outras operações, como soma, subtração, multiplicação e divisão.

```
x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)
print("x = \n", x)
print("y = \n", y)
print("x+y = \n", x+y) # soma
print("x-y = \n", x-y) # subtração
print("x*y = \n", x*y) # multiplicação
print("x/y = \n", x/y) # divisão
```

Código-fonte 6 – Outras operações matemáticas com arrays NumPy
Fonte: Elaborado pelo autor (2023)

```
x =
 [[1. 2.]
 [3. 4.]]
y =
 [[5. 6.]
 [7. 8.]]
x+y =
 [[ 6.  8.]
 [10. 12.]]
x-y =
 [[-4. -4.]
 [-4. -4.]]
x*y =
 [[ 5. 12.]
 [21. 32.]]
x/y =
 [[0.2        0.33333333]
 [0.42857143 0.5       ]]
```

Figura 15 – Resultado do código-fonte 6
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Algo muito útil que podemos fazer com arrays NumPy é alterar sua dimensão, reorganizando os dados na memória. Veja o exemplo da Figura 16. Todos os arrays possuem 12 posições, mas estão acomodados de 6 formas diferentes em termos de linhas e colunas.

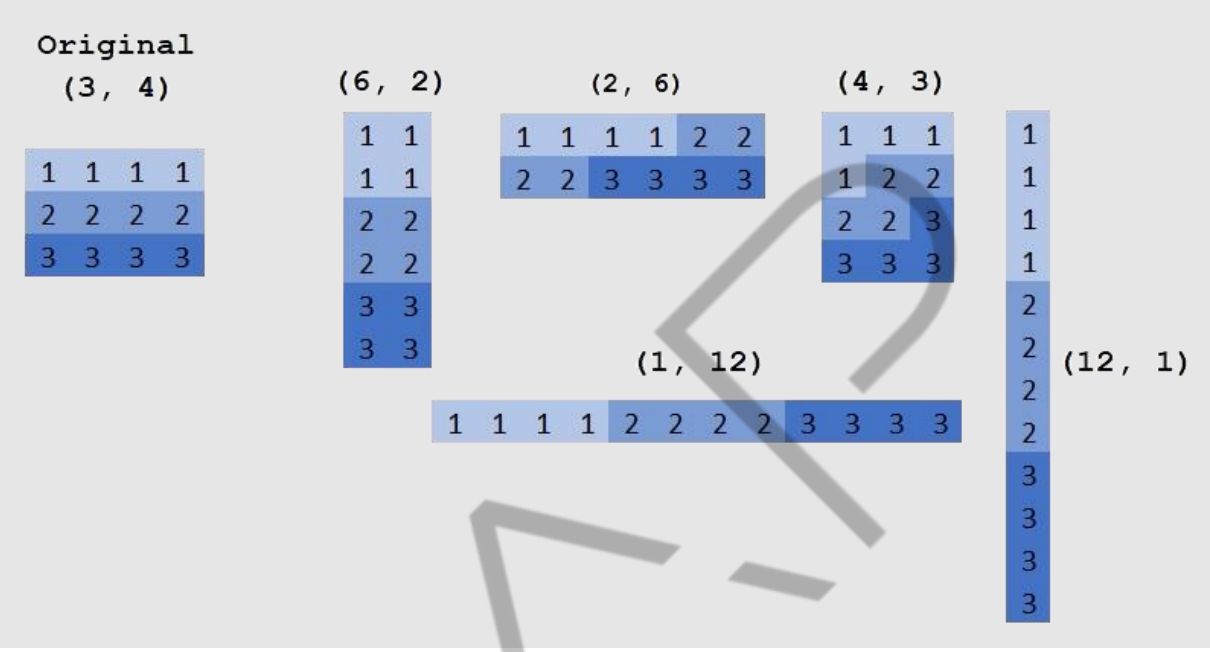

Figura 16 – Alterando tamanhos de arrays NumPy
Fonte: Reproduzida de https://fgnt.github.io/python_crashkurs_doc/include/numpy.html

Vamos ver um exemplo no código abaixo, que termina, inclusive, levando o array de duas dimensões para três dimensões (isso é particularmente interessante considerando que uma imagem é uma matriz tridimensional):

```
array = np.arange(8)
print("array original: \n", array)

array = np.arange(8).reshape(2, 4)
print("\nreshaped 2x4: \n", array)

array = np.arange(8).reshape(4, 2)
print("\nreshaped 4x2: \n", array)

array = np.arange(8).reshape(2, 2, 2)
print("\nreshaped 3D: \n", array)
```

Código-fonte 7 – Alterando tamanhos de arrays NumPy
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
array original:
 [0 1 2 3 4 5 6 7]

reshaped 2x4:
 [[0 1 2 3]
 [4 5 6 7]]

reshaped 4x2:
 [[0 1]
 [2 3]
 [4 5]
 [6 7]]

reshaped 3D:
 [[[0 1]
  [2 3]]

 [[4 5]
  [6 7]]]
```

Figura 17 – Resultado do código-fonte 7
Fonte: Elaborado pelo autor (2023)

Mas afinal, se temos outras estruturas de dados do próprio Python, como as listas, qual é a vantagem de armazenar números (e processá-los) no formato de arrays NumPy? A razão é velocidade! O código abaixo faz a mesma operação diretamente em listas e depois em array NumPy. O tempo de execução é muito menor em NumPy:

```
L = range(1000)              # Cria 1000 números (de 0 a 999)
%timeit [i**2 for i in L]    # Cria uma lista elevando cada um dos 1000
números ao quadrado

a = np.arange(1000)          # Cria um array NumPy denominado "a" com
1000 números de 0 a 999
%timeit a**2                 # Cria um array NumPy que eleva cada valor
de "a" ao quadrado
```

Código-fonte 8 – Comparando tempo de execução de listas Python com arrays NumPy
Fonte: Elaborado pelo autor (2023)

```
361 µs ± 4.85 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
1.55 µs ± 483 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

Figura 18 – Resultado do código-fonte 8
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

5.2 Pandas

O Pandas é uma biblioteca em Python dedicada à manipulação e análise de dados estruturados, oferecendo estruturas de dados flexíveis e eficientes, como o DataFrame. Sua principal utilização reside na organização, limpeza e transformação de dados, simplificando tarefas como filtragem, agregação e indexação.

No contexto do Deep Learning, o Pandas desempenha um papel crucial na preparação de conjuntos de dados, permitindo aos cientistas de dados formatar e estruturar informações de maneira acessível para alimentar redes neurais artificiais sobre dados estruturados.

A habilidade do Pandas em lidar com dados tabulares, bem como sua integração fluida com outras bibliotecas como NumPy, fazem dele uma ferramenta indispensável no ciclo de vida de projetos de ciência de dados.

As estruturas de dados típicas da biblioteca Pandas estão ilustradas na Figura 19. Essencialmente, os dados estão em tabelas denominadas DataFrames. Cada coluna de um DataFrame é uma Series, que essencialmente é um array NumPy indexado por índices.

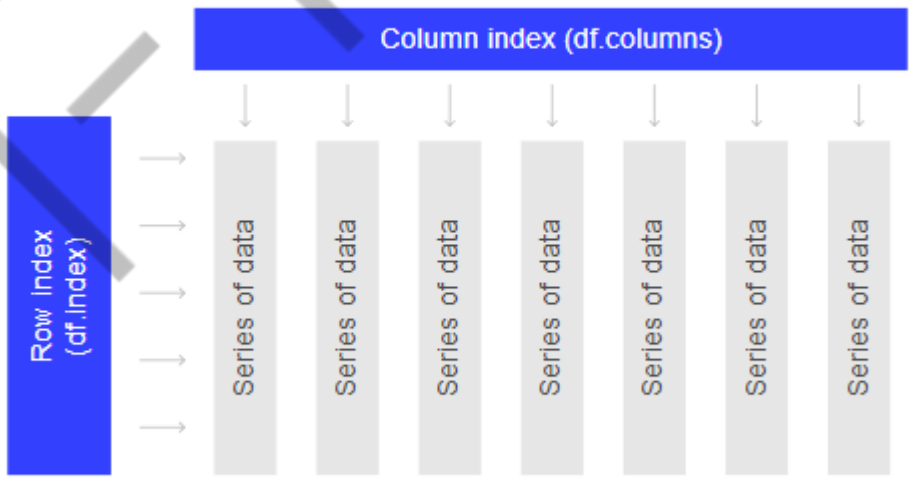

Figura 19 – Estruturas de dados da biblioteca Pandas
Fonte: Reproduzida de https://bookdata.readthedocs.io/en/latest/base/01_pandas.html

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Vamos importar a biblioteca Pandas e conhecer aos poucos suas estruturas de dados. Para começar, vamos criar uma série (Series) qualquer:

```
import pandas as pd
obj = pd.Series( [4, 7, -5, 3] )
obj
```

Código-fonte 9 – Criando uma Series do Pandas
Fonte: Elaborado pelo autor (2023)

O comando acima produz a saída ilustrada na Figura 20. Podemos entender que uma Series do Pandas é um conjunto de valores de mesmo tipo (nesse caso, inteiros), indexados por valores (também inteiros).

```
0    4
1    7
2   -5
3    3
dtype: int64
```

Figura 20 – Resultado do código-fonte 9
Fonte: Elaborado pelo autor (2023)

A partir de uma série do Pandas, podemos ter acesso ao array de valores e de índices:

```
print(obj.values) # array de valores
print(obj.index)  # array de rótulos
```

Código-fonte 10 – Criando uma Series do Pandas
Fonte: Elaborado pelo autor (2023)

O segundo print nos mostra que o range de valores dos índices são inteiros que começam em 0 e terminam em 4, pulando de 1 em 1. Temos, portanto, 5 elementos nessa série de dados.

```
[ 4  7 -5  3]
RangeIndex(start=0, stop=4, step=1)
```

Figura 21 – Resultado do código-fonte 10
Fonte: Elaborado pelo autor (2023)

É possível criar uma Series com um índice que identifique cada ponto de dado com um rótulo, não necessariamente inteiro. Dessa forma:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
obj2 = pd.Series([4, -7, 5, 3], index=["d", "b", "a", "c"])
obj2
```

Código-fonte 11 – Criando uma Series do Pandas indexada por letras
Fonte: Elaborado pelo autor (2023)

O comando acima produz uma Series de 4 elementos, sendo que cada um é indexado por uma letra.

```
d    4
b   -7
a    5
c    3
dtype: int64
```

Figura 22 – Resultado do código-fonte 11
Fonte: Elaborado pelo autor (2023)

Assim, é possível indexar qualquer linha como se fosse um dicionário, com chaves e valores. Os índices são as chaves, e os valores são os dados armazenados:

```
obj2["b"]
```

Código-fonte 12 – Indexando uma Series do Pandas indexada por letras
Fonte: Elaborado pelo autor (2023)

O comando acima deve retornar o valor “-7”. Por estarmos dentro de um ambiente colaborativo, é capaz de vermos o retorno mesmo sem o comando “print()”.

```
-7
```

Figura 23 – Resultado do código-fonte 12
Fonte: Elaborado pelo autor (2023)

O poder do Pandas surge na possibilidade de realizarmos comandos condicionais e filtros nos dados:

```
print(obj2 > 0)          # condicionais
print()
print(obj2[ obj2 < 0 ])  # filtro
```

Código-fonte 13 – Condicionais e filtros no Pandas
Fonte: Elaborado pelo autor (2023)

Observem na Figura 23 as saídas dos prints acima. O primeiro comando retorna, para cada linha da Série, se o valor armazenado nela é maior que zero ou

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

não. Portanto, temos como retorno outra Series, com os mesmos índices da Series original, mas os valores são booleanos (True ou False), que indicam se o dado originalmente armazenado daquela posição atente o comando condicional ou não. Aqui, vemos que todas as linhas, exceto aquela indexada por “b”, são maiores que zero.

Por outro lado, o Pandas também permite fazer filtros. O segundo print usa o retorno do comando condicional para indexar a própria variável “obj2”. Portanto, queremos saber quais são as linhas cujos valores são menores que zero. E o único valor é o “-7”, que está sob o índice “b”.

```
d     True
b    False
a     True
c     True
dtype: bool

b   -7
dtype: int64
```

Figura 24 – Resultado do código-fonte 13
Fonte: Elaborado pelo autor (2023)

Agora que entendemos o que são as Series, podemos construir o tipo de dado central do Pandas: o DataFrame. O DataFrame pode ser construído a partir de Series de dados nomeadas, e com os mesmos índices.

```
ser1 = pd.Series( [1,2,3,4,5], index=list("abcde") )
ser2 = pd.Series( [11,22,33,44,55], index=list("abcde") )
df = pd.DataFrame( {"A": ser1, "B": ser2} )
df
```

Código-fonte 14 – Condicionais e filtros no Pandas
Fonte: Elaborado pelo autor (2023)

O resultado é muito direto: o DataFrame armazenado na variável “df” possui 2 colunas, “A” e “B”, ambas indexadas pelas letras “a”, “b”, “c”, “d” e “e”.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	A	B
a	1	11
b	2	22
c	3	33
d	4	44
e	5	55

Figura 25 – Resultado do código-fonte 14
Fonte: Elaborado pelo autor (2023)

Com o DataFrame em mãos podemos descobrir facilmente seus índices e colunas. Também é trivial obter os dados de uma coluna específica. Pense que uma coluna pode ser uma variável dos seus dados.

```
print(df.index)    # lista de índices
print()
print(df.columns)  # nomes das colunas
print()
print(df["A"])     # indexando a coluna "A"
```

Código-fonte 15 – Obtendo valores do DataFrame
Fonte: Elaborado pelo autor (2023)

Os prints trazem os resultados que esperaríamos ver. A novidade está no tipo de dado “object”, o qual é a forma do Pandas indicar que se trata de uma string.

```
Index(['a', 'b', 'c', 'd', 'e'], dtype='object')

Index(['A', 'B'], dtype='object')

a    1
b    2
c    3
d    4
e    5
Name: A, dtype: int64
```

Figura 26 – Resultado do código-fonte 15
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O Pandas também nos permite criar novas colunas de uma forma muito trivial. Basta indicar o nome da nova coluna e a lista de valores associados a ela. Ele já entende que deve manter os índices do resto do DataFrame. Abaixo, vamos criar uma nova coluna “C”, resultante da soma das colunas “A” e “B”:

```
df["C"] = df["A"] + df["B"]
df
```

Código-fonte 16 – Criando novas colunas em um DataFrame Pandas
Fonte: Elaborado pelo autor (2023)

Como esperávamos, agora o DataFrame “df” contém 3 colunas, nomeadas “A”, “B” e “C”. Os índices continuam sendo os mesmos, nomeados de “a” até “e”.

	A	B	C
a	1	11	12
b	2	22	24
c	3	33	36
d	4	44	48
e	5	55	60

Figura 27 – Resultado do código-fonte 16
Fonte: Elaborado pelo autor (2023)

O Pandas também nos permite criar novas colunas de uma forma muito trivial. Basta indicar o nome da nova coluna e a lista de valores associados a ela. Ele já entende que deve manter os índices do resto do DataFrame. Abaixo, vamos criar uma nova coluna “C”, resultante da soma das colunas “A” e “B”:

```
df["C"] = df["A"] + df["B"]
df
```

Código-fonte 17 – Criando novas colunas em um DataFrame Pandas
Fonte: Elaborado pelo autor (2023)

Como esperávamos, agora o DataFrame “df” contém 3 colunas, nomeadas “A”, “B” e “C”. Os índices continuam sendo os mesmos, nomeados de “a” até “e”.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	A	B	C
a	1	11	12
b	2	22	24
c	3	33	36
d	4	44	48
e	5	55	60

Figura 28 – Resultado do código-fonte 16
Fonte: Elaborado pelo autor (2023)

É claro que no trabalho real como Cientista de dados não vamos ficar criando as Series e DataFrames na mão. Eles são criados automaticamente ao carregarmos dados de arquivos (geralmente CSVs). Veremos adiante que Pandas possui métodos simples de ler e exportar dados em arquivos.

5.3 Explorando e limpando dados com Pandas

DICA: Para essa parte do código, utilize o arquivo de apoio "exemplos_limpeza_dados.csv".

Explorar e limpar dados são fases críticas na análise, e o Pandas destaca-se nessas tarefas. Mergulharemos com Pandas na exploração de um conjunto de dados simples, mas que ilustra os passos de exploração e limpeza de dados.

O código baixo carrega para a memória um DataFrame Pandas contendo os dados do arquivo "exemplos_limpeza_dados.csv". Na sequência, pedimos o formato deste DataFrame, bem como sua exibição.

```
import pandas as pd
import numpy as np

df = pd.read_csv("exemplos_limpeza_dados.csv")
print(df.shape)
print()
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
df
```

Código-fonte 18 – Leitura de um arquivo de dados com Pandas
Fonte: Elaborado pelo autor (2023)

A saída do comando “shape” é uma tupla, indicando a quantidade de linhas e colunas do arquivo (12 linhas e 5 colunas). Após pularmos uma linha, vemos o conteúdo do DataFrame. É importante notar que somente conseguimos ver o DataFrame inteiro porque ele é pequeno! Para dimensões maiores, a visualização seria restrita.

```
(12, 5)
```

	Country	Age	Salary	Grade	Purchased
0	France	44.0	72000.0	1st	No
1	France	44.0	72000.0	1st	No
2	Spain	27.0	48000.0	1st	Yes
3	Germany	30.0	54000.0	3th	No
4	Spain	38.0	61000.0	1st	No
5	Germany	40.0	NaN	2nd	Yes
6	Germany	40.0	NaN	2nd	Yes
7	France	35.0	58000.0	2nd	Yes
8	Spain	NaN	52000.0	1st	No
9	France	48.0	79000.0	3th	Yes
10	Germany	50.0	83000.0	2nd	No
11	France	37.0	67000.0	3th	Yes

Figura 29 – Resultado do código-fonte 17
Fonte: Elaborado pelo autor (2023)

Inclusive, nos casos que temos muitas linhas, podemos alterar a visualização puxando apenas as primeiras linhas (com o head()) ou as últimas (com o tail()). Por via de regra, a melhor representação é feita pelo “sample(N)”, que puxa uma amostra de N linhas aleatórias. Essa é uma representação mais fiel da distribuição de dados

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

do DataFrame inteiro. Abaixo, vamos exibir 5 linhas aleatórias. Sua saída provavelmente será diferente. Aproveite para explorar outros valores para N.

```
# Pegando uma amostra de 5 linhas aleatórias
df.sample(5)
```

Código-fonte 19 – Exibição de 5 linhas aleatórias
Fonte: Elaborado pelo autor (2023)

A saída do comando “shape” é uma tupla, indicando a quantidade de linhas e colunas do arquivo (12 linhas e 5 colunas). Após pularmos uma linha, vemos o conteúdo do DataFrame. É importante notar que somente conseguimos ver o DataFrame inteiro porque ele é pequeno! Para dimensões maiores, a visualização seria restrita.

	Country	Age	Salary	Grade	Purchased
3	Germany	30.0	54000.0	3th	No
11	France	37.0	67000.0	3th	Yes
7	France	35.0	58000.0	2nd	Yes
9	France	48.0	79000.0	3th	Yes
0	France	44.0	72000.0	1st	No

Figura 30 – Resultado do código-fonte 18
Fonte: Elaborado pelo autor (2023)

Parte relevante da exploração da base é compreender quais variáveis existem, seus tipos e a existência de dados nulos. Tudo isso pode ser obtido por meio do comando abaixo:

```
# Explorando a base
df.info()
```

Código-fonte 20 – Obtendo informações das variáveis do DataFrame Pamdas
Fonte: Elaborado pelo autor (2023)

A saída do comando “info()” é muito completa! Ela aponta que estamos diante de um DataFrame Pandas, que ocupa ~608 bytes na memória, temos 5 variáveis (e quais são elas), sabemos que 2 variáveis (Age e Salary) são numéricas do tipo float e

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

as demais são strings. Sabemos, ainda, que o DataFrame possui 12 entradas (linhas), numeradas de 0 a 11. Inclusive, a maior parte das variáveis está completa, havendo valores nulos apenas nas colunas Age, Salary e Grade.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Country     12 non-null     object
 1   Age         11 non-null     float64
 2   Salary      10 non-null     float64
 3   Grade       12 non-null     object
 4   Purchased   12 non-null     object
dtypes: float64(2), object(3)
memory usage: 608.0+ bytes
```

Figura 31 – Resultado do código-fonte 18
Fonte: Elaborado pelo autor (2023)

Inclusive, uma outra forma mais visual de verificar se há dados faltantes é por meio da biblioteca “missingno”. Experimente seu uso com os comandos abaixo:

```
import missingno as msno
msno.matrix(df);
```

Código-fonte 21 – Visualizando dados faltantes com a biblioteca missingno
Fonte: Elaborado pelo autor (2023)

Essa visualização nos permite afirmar que há dados faltantes apenas nas colunas Age e Salary, e que as ausências não ocorrem nas mesmas linhas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

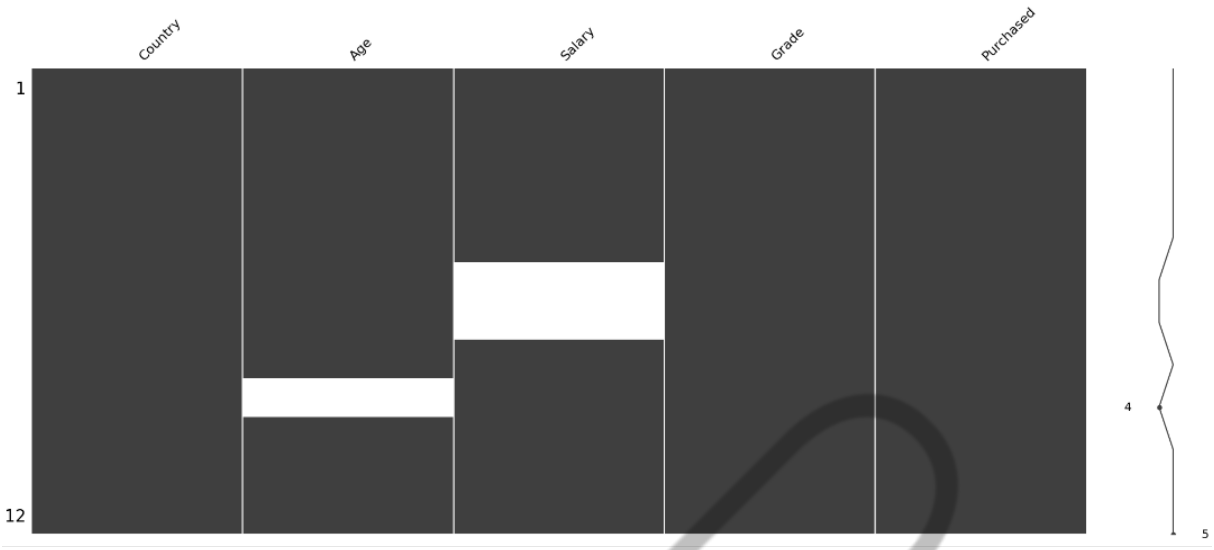

Figura 32 – Resultado do código-fonte 19
Fonte: Elaborado pelo autor (2023)

Outra análise exploratória relevante é obtermos estatísticas básicas das variáveis numéricas. Podemos fazer isso na mão, com poucas linhas de código, mas o Pandas já facilita esse cálculo por meio do comando “describe()”:

```
# Exibe estatísticas básicas do dataframe
df.describe()
```

Código-fonte 22 – Obtendo análises estatísticas básicas do Pandas
Fonte: Elaborado pelo autor (2023)

Essa análise nos mostra, para cada variável numérica, a contagem de linhas não nulas, a média (mean), desvio padrão (std), os valores mínimo (min) e máximo (max), além dos quartis (25%, 50% e 75%). Observe a média da idade (Age) e a mediana do salário (Salary): usaremos esses valores logo mais!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	Age	Salary
count	11.000000	10.000000
mean	39.363636	64600.000000
std	7.060775	11852.800888
min	27.000000	48000.000000
25%	36.000000	55000.000000
50%	40.000000	64000.000000
75%	44.000000	72000.000000
max	50.000000	83000.000000

Figura 33 – Resultado do código-fonte 20
Fonte: Elaborado pelo autor (2023)

Agora chegou a hora de começarmos a limpar os dados de verdade! Uma análise essencial para Machine Learning é evitar termos dados duplicados no dataset. Podemos remover linhas duplicadas de fora simples com os seguintes comandos:

```
# Remover dados duplicados em nova variável
df_unicos = df.drop_duplicates()

# Remover dados duplicados na MESMA variável
df.drop_duplicates(inplace=True)
```

Código-fonte 23 – Obtendo análises estatísticas básicas do Pandas
Fonte: Elaborado pelo autor (2023)

Ambos os comandos acima fazem essencialmente a mesma coisa. O drop_duplicates() sempre retorna uma cópia do DataFrame sem as cópias extras das linhas que aparecem mais de uma vez. A diferença é que o primeiro comando salva esta cópia na variável “df_unicos”, portanto, a variável “df” fica exatamente como está! Já o comando usando o parâmetro “inplace=True” altera a própria variável “df”.

Podemos seguir com a variável “df_unicos” para mostrar como podemos tratar dados nulos com a exclusão das linhas problemáticas. O primeiro comando abaixo apaga as linhas com nulos na própria variável “df_unicos”.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Além disso, outra estratégia é preencher os valores nulos com valores pertinentes. Por exemplo, preenchemos a coluna “Age” com seu valor médio e a coluna “Salary”, com a mediana.

```
# Apagar dados faltantes na MESMA variável
df_unicos.dropna(inplace=True)

# Preencher dados faltantes
# - coloca o valor médio da idade na coluna Age
df["Age"].fillna( df["Age"].mean() , inplace=True)

# - coloca o valor mediano do salário na coluna Salary
df["Salary"].fillna( df["Salary"].median() , inplace=True )
```

Código-fonte 24 – Limpeza de dados com Pandas
Fonte: Elaborado pelo autor (2023)

Reveja os valores obtidos no “describe()” e compare com os valores que apareceram no DataFrame da variável “df”. Compare os estados atuais de “df_unicos” e “df”.

```
df_unicos
```

	Country	Age	Salary	Grade	Purchased
0	France	44.0	72000.0	1st	No
2	Spain	27.0	48000.0	1st	Yes
3	Germany	30.0	54000.0	3th	No
4	Spain	38.0	61000.0	1st	No
7	France	35.0	58000.0	2nd	Yes
9	France	48.0	79000.0	3th	Yes
10	Germany	50.0	83000.0	2nd	No
11	France	37.0	67000.0	3th	Yes

```
df
```

	Country	Age	Salary	Grade	Purchased
0	France	44.000000	72000.0	1st	No
2	Spain	27.000000	48000.0	1st	Yes
3	Germany	30.000000	54000.0	3th	No
4	Spain	38.000000	61000.0	1st	No
5	Germany	40.000000	61000.0	2nd	Yes
7	France	35.000000	58000.0	2nd	Yes
8	Spain	38.777778	52000.0	1st	No
9	France	48.000000	79000.0	3th	Yes
10	Germany	50.000000	83000.0	2nd	No
11	France	37.000000	67000.0	3th	Yes

Figura 34 – Resultado do código-fonte 22
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

5.4 Engenharia de features com Pandas e Scikit-learn

DICA: Continuaremos utilizando o arquivo de apoio "exemplos_limpeza_dados.csv".

Para essa parte do código, certifique-se que as variáveis "df" e "df_unicos" estejam conforme ilustrado na Figura 34.

A engenharia de features é essencial na construção de modelos de aprendizado de máquina robustos, e a combinação do Scikit-Learn e Pandas oferece uma abordagem poderosa nesse processo.

O Scikit-Learn é crucial para Ciência de dados e Deep Learning, proporcionando uma ampla gama de ferramentas e algoritmos de aprendizado de máquina. Sua biblioteca abrangente simplifica a implementação de modelos, oferecendo suporte desde o pré-processamento dos dados até a avaliação do desempenho do modelo. Veremos usos mais sofisticados desta biblioteca adiante.

Por enquanto, utilizaremos a parte de pré-processamento do Scikit-learn para transformar dados categóricos nominais e ordinais, bem como escalar valores numéricos.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.preprocessing import LabelEncoder

# Tratando dados categóricos - ORDINAIS
print( df["Purchased"].unique() ) # valores únicos

le = LabelEncoder()

df["Purchased_encoder"] = le.fit_transform( df["Purchased"] )
df["Grade_encoder"] = le.fit_transform( df["Grade"] )
df
```

Código-fonte 25 – Tratando dados categóricos ordinais
Fonte: Elaborado pelo autor (2023)

O código acima mostra os valores únicos de "Purchased" são "No" e "Yes". Claramente, trata-se de um dado categórico. Da mesma forma, os valores únicos de "Grade" são "1st", "2nd" e "3th". Existe uma relação de ordem entre os dados, portanto,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

usamos o LabelEncoder() do scikit-learn. O “fit_transform()” primeiro cria uma relação entre cada categoria e um valor numérico de 0 em diante, e depois aplica tal relação. Veja a saída na Figura 35.

	Country	Age	Salary	Grade	Purchased	Purchased_encoder	Grade_encoder
0	France	44.000000	72000.0	1st	No	0	0
2	Spain	27.000000	48000.0	1st	Yes	1	0
3	Germany	30.000000	54000.0	3th	No	0	2
4	Spain	38.000000	61000.0	1st	No	0	0
5	Germany	40.000000	61000.0	2nd	Yes	1	1
7	France	35.000000	58000.0	2nd	Yes	1	1
8	Spain	38.777778	52000.0	1st	No	0	0
9	France	48.000000	79000.0	3th	Yes	1	2
10	Germany	50.000000	83000.0	2nd	No	0	1
11	France	37.000000	67000.0	3th	Yes	1	2

Figura 35 – Resultado do código-fonte 23
Fonte: Elaborado pelo autor (2023)

Agora vamos tratar as variáveis categóricas nominais, ou seja, aquelas que não possuem relação de ordem entre cada categoria. Nesse caso, usamos a técnica de *One Hot Encoder*, que é implementada, por exemplo, pelo método “get_dummies()” do Pandas.

```
# Tratando dados categóricos - NOMINAIS
aux = pd.get_dummies( df["Country"] )
final = pd.concat( [df, aux], axis=1 )
final
```

Código-fonte 26 – Tratando dados categóricos nominais
Fonte: Elaborado pelo autor (2023)

Note que agora nosso novo DataFrame “final” está bem mais amplo, pois novas features foram criadas automaticamente.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	Country	Age	Salary	Grade	Purchased	Purchased_encoder	Grade_encoder	France	Germany	Spain
0	France	44.000000	72000.0	1st	No	0	0	1	0	0
2	Spain	27.000000	48000.0	1st	Yes	1	0	0	0	1
3	Germany	30.000000	54000.0	3th	No	0	2	0	1	0
4	Spain	38.000000	61000.0	1st	No	0	0	0	0	1
5	Germany	40.000000	61000.0	2nd	Yes	1	1	0	1	0
7	France	35.000000	58000.0	2nd	Yes	1	1	1	0	0
8	Spain	38.777778	52000.0	1st	No	0	0	0	0	1
9	France	48.000000	79000.0	3th	Yes	1	2	1	0	0
10	Germany	50.000000	83000.0	2nd	No	0	1	0	1	0
11	France	37.000000	67000.0	3th	Yes	1	2	1	0	0

Figura 36 – Resultado do código-fonte 24
Fonte: Elaborado pelo autor (2023)

No entanto, claramente não precisamos de todas estas features, pois muitas são cópias transformadas das colunas originais. Então, vamos fazer outra limpeza, removendo colunas que não são mais necessárias. O comando abaixo pode ser feito com qualquer feature que julgamos irrelevante na nossa análise:

```
# Filtrar colunas relevantes
final.drop( columns=["Country", "Grade", "Purchased"] , inplace=True )
final
```

Código-fonte 27 – Como eliminar colunas via Pandas
Fonte: Elaborado pelo autor (2023)

Ao usarmos o parâmetro “inplace=True”, alteramos a própria variável “final”. Ao exibir o resultado temos:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	Age	Salary	Purchased_encoder	Grade_encoder	France	Germany	Spain
0	44.000000	72000.0	0	0	1	0	0
2	27.000000	48000.0	1	0	0	0	1
3	30.000000	54000.0	0	2	0	1	0
4	38.000000	61000.0	0	0	0	0	1
5	40.000000	61000.0	1	1	0	1	0
7	35.000000	58000.0	1	1	1	0	0
8	38.777778	52000.0	0	0	0	0	1
9	48.000000	79000.0	1	2	1	0	0
10	50.000000	83000.0	0	1	0	1	0
11	37.000000	67000.0	1	2	1	0	0

Figura 37 – Resultado do código-fonte 25
Fonte: Elaborado pelo autor (2023)

Ótimo! Agora nossas features estão completamente numéricas, uma exigência dos modelos de aprendizado de máquina e Deep Learning. No entanto, elas se apresentam em escalas bem diferentes! Nosso próximo comando transformará os dados por meio de Padronização e Normalização.

A Padronização (StandardScaler()) é mais voltada para dados que seguem a distribuição normal, ou seja, centrados na média. A Normalização (MinMaxScaler()) apenas leva a distribuição dos dados para outro intervalo de valores (no nosso caso, entre 10 e 20).

```
# Transforma os dados com Padronização/Normalização
sc = StandardScaler()
final_padronizado = sc.fit_transform(final)

mm = MinMaxScaler(feature_range=(10, 20))
final_normalizado = mm.fit_transform(final)
```

Código-fonte 28 – Trazendo dados para a mesma escala
Fonte: Elaborado pelo autor (2023)

Para melhor exibir os dados, criaremos novos DataFrames Pandas. Primeiramente, para os dados padronizados:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Salvar os dados padronizados em outro dataframe
df_final_padronizado = pd.DataFrame(final_padronizado,
                                    columns=final.columns)
df_final_padronizado
```

Código-fonte 29 – Criando um DataFrame com os dados Padronizados
Fonte: Elaborado pelo autor (2023)

E vejamos o resultado!

	Age	Salary	Purchased_encoder	Grade_encoder	France	Germany	Spain
0	0.758874	0.772568	-1.0	-1.083473	1.224745	-0.654654	-0.654654
1	-1.711504	-1.408800	1.0	-1.083473	-0.816497	-0.654654	1.527525
2	-1.275555	-0.863458	-1.0	1.324244	-0.816497	1.527525	-0.654654
3	-0.113024	-0.227226	-1.0	-1.083473	-0.816497	-0.654654	1.527525
4	0.177609	-0.227226	1.0	0.120386	-0.816497	1.527525	-0.654654
5	-0.548973	-0.499897	1.0	0.120386	1.224745	-0.654654	-0.654654
6	0.000000	-1.045239	-1.0	-1.083473	-0.816497	-0.654654	1.527525
7	1.340140	1.408800	1.0	1.324244	1.224745	-0.654654	-0.654654
8	1.630773	1.772361	-1.0	0.120386	-0.816497	1.527525	-0.654654
9	-0.258340	0.318116	1.0	1.324244	1.224745	-0.654654	-0.654654

Figura 38 – Resultado do código-fonte 27
Fonte: Elaborado pelo autor (2023)

Agora, façamos o mesmo com os dados Normalizados:

```
# Salvar os dados normalizados em outro dataframe
df_final_normalizado = pd.DataFrame(final_normalizado,
                                    columns=final.columns)
df_final_normalizado
```

Código-fonte 30 – Criando um DataFrame com os dados Normalizados
Fonte: Elaborado pelo autor (2023)

E vejamos o resultado!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	Age	Salary	Purchased_encoder	Grade_encoder	France	Germany	Spain
0	17.391304	16.857143	10.0	10.0	20.0	10.0	10.0
1	10.000000	10.000000	20.0	10.0	10.0	10.0	20.0
2	11.304348	11.714286	10.0	20.0	10.0	20.0	10.0
3	14.782609	13.714286	10.0	10.0	10.0	10.0	20.0
4	15.652174	13.714286	20.0	15.0	10.0	20.0	10.0
5	13.478261	12.857143	20.0	15.0	20.0	10.0	10.0
6	15.120773	11.142857	10.0	10.0	10.0	10.0	20.0
7	19.130435	18.857143	20.0	20.0	20.0	10.0	10.0
8	20.000000	20.000000	10.0	15.0	10.0	20.0	10.0
9	14.347826	15.428571	20.0	20.0	20.0	10.0	10.0

Figura 39 - Resultado do código-fonte 28
Fonte: Elaborado pelo autor (2023)

Pronto! Agora sim os dados estão todos no formato esperado pela máquina para aprender a encontrar padrões nos dados e treinar algoritmos de machine learning. Note que todos os dados são numéricos e estão na mesma escala. Os DataFrames finais “df_final_padronizado” ou “df_final_normalizado” podem ser usados para alimentar algoritmos de machine learning.

A título de curiosidade, podemos, ainda, exportar os dados salvos em arquivos CSV pelos seguintes comandos.

```
# Exporta os dados para arquivos
df_final_padronizado.to_csv("dados_padronizados.csv")
df_final_normalizado.to_csv("dados_normalizados.csv")
```

Código-fonte 31 – Exportando o resultado das transformações aplicadas
Fonte: Elaborado pelo autor (2023)

Após executá-los, note a criação de dois novos arquivos no seu ambiente de execução do Google Colaboratory (Figura 40).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

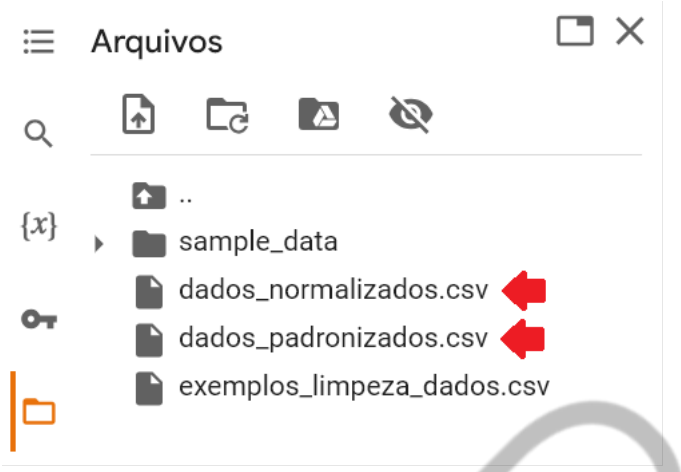

Figura 40 – Exibição de novos arquivos criados no ambiente Google Colab
Fonte: Elaborado pelo autor (2023)

6 CONCLUSÃO

Ao longo deste capítulo, exploramos as diversas dimensões do Universo dos Dados, desde a classificação baseada em fonte, granularidade, estrutura e tipo até sua organização em bancos de dados (e no Big data).

Adentramos o fascinante domínio da Ciência de Dados, abordando a manipulação de variáveis, técnicas de amostragem e as etapas cruciais de pré-processamento e engenharia de features. Descobrimos e compreendemos os desafios inerentes ao trabalho com dados, como overfitting e underfitting, contrastando-os com o perfil do “modelo ideal”.

Na parte prática, mergulhamos nas bibliotecas NumPy e Pandas, destacando sua grande importância na análise e manipulação de dados. Ao incorporar técnicas de exploração e limpeza de dados, além da engenharia de features com Pandas e Scikit-Learn, capacitamos os leitores a não apenas compreender, mas também aplicar habilidades fundamentais de Ciência de dados em projetos do mundo real. Essa atividade prática reforça o aprendizado teórico, fornecendo uma base sólida para explorar os vastos horizontes da modelagem de dados na era da informação.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

BIBLIOTECA NumPy. Disponível em: <https://numpy.org/>. Acesso em: 24 nov. 2023.

BIBLIOTECA Pandas. Disponível em: <https://pandas.pydata.org/>. Acesso em: 24 nov. 2023.

CIELEN, Davy; MEYSMAN, Arno. **Introducing data science:** big data, machine learning, and more, using Python tools. Simon and Schuster, 2016.

PROVOST, F. **Data Science para Negócios** by Tom Fawcett Foster Provost [Fawcett, Tom](z-lib. org).

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

GLOSSÁRIO

AI	*Artificial Intelligence*
DL	*Deep Learning*
IA	Inteligência Artificial
ML	*Machine Learning*
RNA	Redes Neurais Artificiais
SGBD	Sistema de Gerenciamento de Banco de Dados

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com